

PROJECT DOCUMENTATION · KNOWLEDGE BASE SOURCE

Aixia

A personal retrieval-augmented generation chatbot.
Technical and non-technical reference.

Author Sean Vincent Vien Vinas

Live aixia-project.vercel.app

Stack Next.js · Django REST Framework · LangChain · pgvector · Groq · Gemini

Hosting Vercel · Render · Supabase

Document version 2.0 — 7 September 2026

Intended use Human reference and RAG ingestion corpus

1. What Aixa Is

Aixa is a full-stack retrieval-augmented generation (RAG) chatbot that answers questions about Sean Vincent Vien Vinas's professional background. It answers only from PDFs uploaded to it — a CV and related documents — and shows the source excerpts behind every answer. It is a portfolio project, built to demonstrate end-to-end RAG engineering, and it is live in production.

- **Live frontend** — <https://aixa-project.vercel.app> (Vercel)
- **Live backend** — <https://aixa-backend.onrender.com> (Render, Docker, free plan, Singapore)
- **Went live** — 6 September 2026
- **Repository** — Django backend in `backend/`, Next.js frontend in `frontend/`

Why it exists

A CV is static; a reader cannot ask it a follow-up question. Aixa turns one into something interrogable while keeping every answer anchored to the document rather than to a language model's memory. The project deliberately bridges classical ML and embedded engineering experience with modern LLM application development.

The core guarantee: grounded, not guessed

The prompt instructs the model to answer using only retrieved context and to reply "I don't have that information" when the answer is absent. A general-purpose chatbot asked about a private individual will confidently fabricate; Aixa is built to refuse instead. The visible sources panel is the other half of that promise — a sceptical reader can check any claim without leaving the page.

What it deliberately does not do

Aixa has no login for chat; anyone who can reach the page can ask. Only document upload and ingestion require a Django administrator. Conversations are not shared between devices — the chat list lives in that browser's `localStorage`. Only PDFs are accepted, up to 20 MB, and there is no OCR, so a scanned image-only PDF yields little text.

How RAG works here, briefly

Retrieval-augmented generation means finding the passages relevant to a question and pasting them into the model's prompt, so the model is never asked what it knows — it is asked to answer from supplied text. Aixa implements both halves: retrieval turns PDFs into a searchable vector index and finds matching passages; generation hands those passages plus the question to Groq and streams the answer back. An **embedding** is a vector representing meaning, so a search for "machine learning experience" matches "trained a convolutional neural network" despite sharing no words. A **chunk** is a passage cut from a document before indexing, because a whole CV is too coarse a unit to retrieve and too wasteful to paste wholesale.

2. Architecture

Runtime pieces

- **Next.js frontend** (Vercel) — renders the chat UI and server-side proxies every `/api/*` path to Django, so the browser never learns the backend's hostname and no backend URL is baked into the browser bundle.
- **Django + DRF backend** (Render, Docker) — HTTP API, session and message persistence, and the orchestration of retrieval and generation.
- **Supabase Postgres** (ap-southeast-1) — one database holding both relational data (documents, sessions, messages) and the pgvector embedding rows.
- **Supabase Storage** — S3-protocol object storage for uploaded PDFs.
- **Groq** — hosted LLM inference for answers, follow-up suggestions, and conversation titles.
- **Gemini** — hosted embeddings for indexing and querying.

Two decoupled paths

The Django apps `documents` and `chat` own everything relational. The `backend/apps/rag/` package owns retrieval and generation and imports no Django at all, so it can be tested with fake models and no database. The two meet in exactly two places: `documents/b_services.py`, which runs a `Document` row through the pipeline, and `chat/b_views.py`, which calls the answering functions.

Request lifecycle: a streamed chat turn

The browser POSTs to `/api/chat/stream/`; Next.js rewrites it to Django. The backend validates the body, resolves or creates a session (verifying a signed token if a session id is claimed), loads up to the last 20 messages as history, and stores the question. It embeds the question via Gemini, retrieves the top 3 chunks from pgvector, builds the prompt, and opens a streaming Groq call. It returns a `StreamingHttpResponse` of newline-delimited JSON with `Cache-Control: no-cache` and `X-Accel-Buffering: no`. Sources are emitted first, before a single token exists, then tokens as they arrive; the assembled answer is persisted, then suggestions and (on the opening turn) a title are generated, and finally a `done` event carries the session id and a fresh signed token.

Request lifecycle: ingestion

An administrator uploads a PDF, creating a `Document` row with `is_ingested=false`. Ingestion is a separate, explicit step: read the file through Django's storage API (never `.path`, which raises on S3), load pages with PyMuPDF, split into chunks, stamp each chunk with `document_id` and `original_filename`, embed via Gemini, write to pgvector, and flip `is_ingested`. It is synchronous — the request stays open for the whole embedding pass.

3. Technology Stack

- **Frontend** — Next.js 16.2.10, React 19.2.4, JavaScript. Plain CSS in `app/globals.css` with semantic class names, ported verbatim from the design mockup; Tailwind is no longer used for component styling. Fonts: Abril Fatface, Comfortaa, Inter, IBM Plex Mono.
- **Backend** — Django 6.0.7, Django REST Framework 3.17.1, Python 3.12, Gunicorn 26.0.0, WhiteNoise 6.12.0, django-cors-headers 4.9.0, python-dotenv 1.1.1.
- **Database driver** — psycopg[binary] 3.3.5. Psycopg 3 only: Django 6 prefers it when both are present and langchain-postgres requires it, so shipping one driver avoids ambiguity.
- **RAG** — langchain 1.3.11, langchain-core 1.6.2, langchain-groq 1.1.3, langchain-text-splitters 1.1.2, langchain-google-genai 4.4.0, langchain-postgres 0.0.17.
- **PDF** — PyMuPDF 1.28.0, called directly.
- **Storage** — django-storages[s3] 1.14.6 against Supabase Storage.
- **LLM** — Groq serving qwen/qwen3.6-27b, temperature 0, reasoning disabled.
- **Embeddings** — Gemini models/gemini-embedding-001 at 768 dimensions.

4. The RAG Pipeline

Loading — `rag/a_loader.py`

`load_document(data: bytes, source_name)` opens a PDF from raw bytes with PyMuPDF and returns one LangChain Document per page, carrying source, page (0-based), and `total_pages` metadata.

It takes bytes rather than a path because uploads live in object storage, where `FileField.path` raises `NotImplementedError`; bytes is the one input that works for both local disk and S3. PyMuPDF is called directly rather than through langchain-community's loader because that package is being sunset upstream and was the only thing the project imported from it — ten lines of reimplementation drops the whole dependency.

Splitting — `rag/b_splitter.py`

A `RecursiveCharacterTextSplitter` with separators `["\n\n", "\n", ". ", " ", ""]`, `CHUNK_SIZE` 2000 and `CHUNK_OVERLAP` 250, both overridable by environment variable.

2000 was measured, not chosen by taste. Google's embedding quota counts every chunk as one request (100/minute), not every batched call, so halving the chunk count halves the quota cost of ingesting a document. At 2000 the largest real document needs 79 chunks, leaving 21 of headroom. At 1500 the same document needed 102 — over the entire per-minute budget on its own, so no amount of retrying could ever have indexed it. The original 500 (roughly 125 tokens) also split sentences mid-thought, costing retrieval quality as well as quadrupling the request count.

Embedding — `rag/c_embeddings.py`

`get_embeddings()` returns `GoogleGenerativeAIEmbeddings` for `models/gemini-embedding-001` with `output_dimensionality=768`.

This is a hosted API call rather than a local sentence-transformers model, and that is a hosting constraint rather than a preference: the local model pulls in torch, roughly 3 GB of image and several hundred MB resident per worker, more than a free-tier container has. Groq offers no embeddings endpoint, so Gemini is a second provider by necessity.

Why 768 dimensions. The model returns 3072 by default, but pgvector's HNSW index refuses anything wider than 2000 — at full width the vectors could be stored but never indexed. Gemini is trained with Matryoshka representation learning, so truncating is supported rather than a lossy hack. Changing this number invalidates every stored vector: embeddings of different widths are not comparable, and the index must be rebuilt from the source PDFs.

Storing and searching — `rag/d_vectorstore.py`

`PGVector` against the application's own Postgres, collection `aixia_documents`, with `embedding_length=768` and `use_jsonb=True`. Declaring the width makes the column `vector(768)` rather than unconstrained, which is what permits an index; without it searches still work but degrade to a full scan. `search(vectorstore, query, k=3)` is the retrieval call.

`connection_string()` rewrites `DATABASE_URL` from Django's `postgres://` to the `postgresql+psycopg://` form SQLAlchemy needs, so one environment variable serves both rather than two that can drift apart.

Why pgvector replaced Chroma. Chroma persisted to a directory on disk — fine locally, quietly broken on a free-tier PaaS box with an ephemeral filesystem. The index would be discarded on every deploy and every wake from sleep with no error to notice; retrieval would simply return nothing. Vectors in Postgres live exactly as long as the database, and one fewer component needs hosting.

Prompting — rag/e_prompts.py

Three templates. `context_prompt` takes history, context, and question; it establishes the grounding rule, then sets a detailed Markdown formatting policy (headings, nested and task lists, tables, fenced code, quotes, rules) and a citation policy: cite inline as `[1]` immediately after the supported sentence, use only chunk numbers that appear, write `[1][2]` rather than `[1, 2]`, and never append a Sources list. `suggestions_prompt` asks for 2–3 follow-up questions as a bare JSON array. `title_prompt` asks for a short conversation title.

Generation — rag/f_chains.py

`get_llm(max_tokens)` builds a ChatGroq client. `answer_question()` is the blocking path; `answer_question_stream()` returns retrieved docs immediately plus a lazy generator of visible text. `format_docs()` numbers chunks 1-based in retrieval order; `format_history()` renders turns as `User: / Assistant: lines`, or "No previous conversation.". Both answering functions accept an injected `llm` so tests can pass a fake.

Token budgets exist because Groq's free tier rejects a request up front when `max_tokens` exceeds what is left of the per-minute ceiling (1000 output tokens per minute). The answer's real length is irrelevant — what matters is the budget reserved, so an old 4096 ceiling was refused with a 429 before the model ran at all. Answers reserve 700, suggestions 200, titles 24. On an opening turn all three share one minute.

Reasoning is turned off (`reasoning_effort="none"`). qwen3.6 is a reasoning model and its `<think>` block is billed as output against that same cap even though it is discarded before the user sees a word — the budget was going on hidden text.

`generate_followup_suggestions()` and `generate_title()` are both best-effort and can never raise: they return empty on a failed call, invalid JSON, or non-list JSON, and run after the answer is already persisted, so a failure loses nothing. Title generation additionally strips the quotes, `Title:` prefixes, and trailing periods models add regardless of instruction.

Stripping reasoning tags — rag/g_stream_filter.py

`_strip_thinking()` in `f_chains.py` handles whole strings with two regexes, the second covering an unclosed tag from truncated output. Streaming cannot use a regex, because a tag may be split across token boundaries, so `strip_thinking_stream(chunks)` is a generator holding back the last `len(tag) - 1` characters of each fragment in case they begin a tag whose remainder has not arrived. It tracks an `in_think` flag and whitespace so the trimmed result matches the regex version exactly. If the stream ends inside a block, the unterminated content is discarded.

5. HTTP API

Chat endpoints are open, throttled by DRF's anonymous rate at 30 requests/minute. Document endpoints require an authenticated Django administrator (IsAdminUser).

POST /api/documents/upload/

Multipart, single file field. Returns 400 for a missing file, a non-.pdf extension, or a file over 20 MB; 201 with the serialized document otherwise.

POST /api/documents/<document_id>/ingest/

No body. 404 if unknown; 200 with "Document already ingested" if the flag is set; 422 if the pipeline raises; otherwise 200 with a chunks_created count.

POST /api/chat/

Blocking. Body: question (required), optional session_id + session_token. Returns session_id, a fresh session_token, the complete answer, and a sources array of {content, document_id, original_filename}. 400 on an empty question, 403 on a bad or missing token for a claimed session, 404 if the signed session no longer exists.

GET /api/chat/

Always 403. Listing history without authentication would let anyone enumerate other people's conversations; this is an acknowledged gap, not an oversight.

POST /api/chat/stream/

Same body plus an optional regenerate flag. Sessions are validated before streaming begins, so a bad token yields an ordinary 403 rather than a stream containing an error. Responds application/x-ndjson, one complete JSON object per line, in six possible event types:

```
{ "type": "sources", "sources": [ {content, document_id, original_filename, page} ] }
{ "type": "token", "content": "..."} # zero or more, in order
{ "type": "suggestions", "suggestions": ["...", "..."]}
{ "type": "title", "title": "..."} # opening turn only
{ "type": "done", "session_id": 1, "session_token": "..."}
{ "type": "error", "message": "..."} # replaces remaining tokens
```

Regeneration. With regenerate: true, question is ignored but a valid session and token are required. Aixa reads the last two messages; if they are not an assistant preceded by a user, it returns 400. Otherwise it deletes the stale answer, reuses the previous question, and trims the trailing history entry — without that trim the question would appear both as the current question and as the last line of history.

GET /healthz/

Liveness probe returning {"status": "ok"}. It deliberately touches neither the database nor any upstream API: this answers "is the process accepting requests", not "is every dependency healthy". A Supabase blip should not convince Render to kill a working container, and a probe costing a DB round trip on a free instance competes with real traffic. It is exempt from the HTTPS redirect, because Render probes over plain HTTP internally and a 301 would read as unhealthy.

Session tokens

Produced with Django's `signing.dumps` under the salt `aixia-chat-session`, signed with the secret key, valid 30 days. A client supplying a `session_id` must supply the matching token; supplying neither yields a new session. This is what stops a visitor reading another's conversation by guessing a sequential integer id.

6. Data Model

- **Document** — `file` (FileField, upload/), `original_filename` (max 120 chars, preserved even if storage renames the stored file), `uploaded_at`, `is_ingested`. The flag makes ingestion idempotent: a second request returns early instead of duplicating chunks.
- **ChatSession** — `created_at` only. No owning user; access control is the signed token.
- **ChatMessage** — `session` FK (cascade, related name `messages`), `role` (user/assistant), `content`, `created_at`.
- **Vector rows** — written by `langchain-postgres` into the same database. Chunk metadata carries `document_id` and `original_filename` (stamped at ingestion) plus `source`, `page`, and `total_pages` (from the loader).

The citation contract

Three pieces of code must agree, and a comment in `f_chains.py` says so. `format_docs()` numbers chunks 1-based in retrieval order; `_serialize_sources()` preserves that order; the frontend treats `[n]` as `sources[n-1]`. Breaking any one silently misattributes citations.

7. Frontend

`ChatWindow.js` (586 lines) is the stateful root: conversations, streaming, theme, and backend status. Each chat holds an `id`, `title`, `messages`, `sessionId`, `sessionToken`, and `updatedAt`. Streaming chats are tracked in a `Set`, not a `boolean`, so switching conversations mid-answer leaves both working. Presentation is split into focused components:

- `Markdown.js` (314 lines) — hand-rolled renderer covering exactly what the answer prompt asks for: headings, nested ordered/unordered and task lists, tables, fenced code, blockquotes, rules, inline emphasis/code/links. Hand-rolled rather than an npm library for two reasons: `[n]` markers must become buttons wired to the sources panel, and it renders **mid-stream**, so every branch must tolerate a half-written document — an unterminated fence renders as code, a ragged table still renders. Only `http(s):` and `mailto:` links are made clickable.
- `typingPacer.js` — Groq returns tokens in lumps, and painting each lump on arrival makes text jump rather than type. The pacer buffers tokens and drains them at a steady rate (30 ms tick, 70 c/s base). Its rate comes from a **deadline** (`pending / secondsRemaining`), not `pending / WINDOW`; the latter is exponential decay that only approaches the base rate and never lands, taking 7.6 s to drain a 3000-character answer.
- `MessageRow.js` — one turn: bubble, copy, regenerate, thumbs up/down, follow-up chips.
- `SourcesPanel.js` — the retrieved excerpts, focus-trapped, Escape to close, scrolling the highlighted source into view when a citation is clicked.
- `ThinkingBubble.js` — rotating wait phrases on the mockup's exact cadence (950 ms rotation, 160 ms fade), starting at a random index so consecutive answers differ.
- `Composer.js` — input, PDF attach (posting to `/api/documents/upload/`, mirroring the backend's 20 MB ceiling), and conversation export.
- `Sidebar.js`, `AppHeader.js`, `icons.js`, `sourceLabel.js` — conversation list with inline rename, header with status pill and theme toggle, shared SVGs, and the one function that names a chunk (`filename · p.N`, converting the loader's 0-based page) for all three places that display one.

Theme

Light is the default; the header toggle persists the choice. **Three places must agree or the palette flashes on load**: the SSR data-theme on `<html>`, the pre-paint bootstrap script, and `ChatWindow`'s theme `useState` initialiser. `themeTint.js` holds the `<meta name="theme-color">` values shared by all three, because Safari tints its status bar from them and they must track `--surface-0` per palette.

CSS gotcha: `--fast` and `--medium` in `globals.css` bundle duration `*and* easing (180ms var(--ease))`.

Writing `var(--medium) var(--ease)` yields two timing functions, which makes the whole declaration invalid — the browser silently drops it and the motion never runs.

Local persistence

`localStorage` keys `aixia-chat-history`, `aixia-sidebar-open`, and `aixia-theme-v2`. Every read and write is wrapped in `try/catch`, since storage can be disabled or full; on failure the UI works in memory for that page load. History is restored after hydration, because browser storage is external state unavailable during server rendering.

Design reference

`aixia-chat-mockup.html` at the repo root is the finalized, authoritative design — two floating glass panels over an ambient grid canvas, in light (cream + gold) and dark (navy + indigo). UI changes are checked against it rather than invented. Deliberate deviations that are intentional: collapsed, the Aixia logo *is* the expand button (the mockup hides its only toggle when collapsed, making collapse irreversible); the status pill sits in the header's right

group; below 60rem an expanded sidebar floats over the thread as a drawer with `.main` carrying `grid-column: 2`; the hero's starter prompts are one row drawn per conversation from a 12-item pool.

8. Configuration

Backend environment

- `DJANGO_SECRET_KEY` — signs session tokens. With `DJANGO_DEBUG=False` and no key, startup raises rather than running insecurely. In debug, a random throwaway key is generated per start, which invalidates existing tokens — local development only.
- `DJANGO_DEBUG` — defaults true.
- `DATABASE_URL` — full PostgreSQL URL; required, since the vector store lives in the same database. Falls back to `DB_NAME/DB_USER/DB_PASSWORD/DB_HOST/DB_PORT` for the relational connection only.
- `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS` — comma-separated. Hosts carry no scheme; CSRF origins must. `RENDER_EXTERNAL_HOSTNAME` is appended automatically, and the LAN IP is detected and added for local network access.
- `CORS_ALLOW_ALL_ORIGINS` — defaults to the value of `DJANGO_DEBUG`.
- `GROQ_API_KEY`, `GROQ_MODEL`, `GROQ_MAX_TOKENS`, `GROQ_SUGGESTION_MAX_TOKENS`, `GROQ_TITLE_MAX_TOKENS`, `GROQ_REASONING_EFFORT`.
- `GOOGLE_API_KEY`, `EMBEDDING_MODEL`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, `PGVECTOR_COLLECTION`.
- `AWS_STORAGE_BUCKET_NAME`, `AWS_S3_ENDPOINT_URL`, `AWS_S3_REGION_NAME`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` — presence of the bucket name switches media to S3; without it uploads land on the container filesystem and are destroyed on the next deploy.

With debug off, Django also enables `SECURE_SSL_REDIRECT` (exempting `/healthz/`), HSTS for a year with subdomains and preload, `SECURE_CONTENT_TYPE_NOSNIFF`, and trusts `HTTP_X_FORWARDED_PROTO` as the TLS signal behind the proxy.

Frontend environment

`BACKEND_INTERNAL_URL` — where the Next.js server proxies `/api/*`. Read server-side only, never inlined into the browser bundle, so changing it needs a restart rather than a rebuild. Defaults to the Docker service name `http://backend:8000`.

Secrets live only in environment variables. `backend/.env`, `frontend/.env.local`, and `deploy.env` are gitignored; only `.env.example` templates are committed. Deployment values are collected by `scripts/deploy-wizard.sh` into `deploy.env`.

9. Running Aixia

Local, with Docker

`docker-compose.yml` defines `db` (`postgres:16-alpine`, `health-checked`), `backend`, and `frontend`. Host ports are `AIXIA_BACKEND_PORT` (default 8001) and `AIXIA_FRONTEND_PORT` (default 3000), so a port collision is resolved by editing the root `.env` with `no rebuild`. `docker-compose.override.yml` bind-mounts `./backend` for live source; `docker-compose.prod.yml` sets restart policies, turns debug off, and passes secrets through.

```
# set GROQ_API_KEY and GOOGLE_API_KEY in backend/.env first
docker compose up --build
docker compose exec backend python manage.py migrate
```

Gunicorn does not auto-reload, so backend edits need `docker compose restart backend` — without it a request succeeds against stale code.

Local frontend dev server

For frontend work use a real dev server with hot reload rather than the built Docker image on port 3000, which goes stale. `.claude/launch.json` defines it on port **3210**. Because that server runs on the host, it cannot resolve the Docker service name `backend`, so `frontend/.env.local` must point at the published host port:

```
BACKEND_INTERNAL_URL=http://localhost:8001
```

Ingesting a document

Upload via the API or the composer's attach control, then run the **Ingest selected documents** action in the Django admin. On the deployed instance the admin action is the only way to index anything at all, because Render's free tier has no shell. Each document is handled independently and reports its own outcome, so one malformed PDF cannot silently abort a batch.

Because Render has no shell, `createsuperuser` and similar are run locally against the production `DATABASE_URL`.

10. Production Deployment

Topology

Frontend on **Vercel** (root directory frontend, needs `BACKEND_INTERNAL_URL`). Backend on **Render** via `render.yaml` Blueprint — Docker runtime, free plan, **singapore** region. Postgres + pgvector and S3-protocol Storage on **Supabase**, `ap-southeast-1`.

Region matters. A chat request makes several database round trips — session lookup, message insert, vector search, message insert — so a cross-Pacific split would add roughly 180 ms to each. Render cannot change a service's region after creation, so it has to be right the first time.

Database URL must be the Supabase session pooler — port 5432 on the `aws-N-<region>.pooler.supabase.com` host, not the direct connection Supabase shows first, which is IPv6-only and unreachable from Render. Session mode rather than transaction mode (6543), because `langchain-postgres` uses `psycopg3`, whose prepared statements break under transaction pooling.

Migrations run in `dockerCommand (/app/start.sh)`, because Render's `preDeployCommand` is paid-tier only. One Unicorn worker rather than the Dockerfile's three: the free tier has 512 MB and each worker is a full copy of the app. Threads rather than processes, because a sync worker is tied up for a whole streamed response and a single-threaded worker would serve one conversation at a time. The timeout is raised because a streamed answer legitimately outlives the 30 s default. `start.sh` is passed as a plain path — Render already runs the command through a shell, so a self-wrapped `sh -c "a && b"` is read as one command name and dies with exit 127.

`DJANGO_SECRET_KEY` uses Render's `generateValue`, so the secret never passes through a clipboard. `ALLOWED_HOSTS` is deliberately absent from the blueprint, since settings appends `RENDER_EXTERNAL_HOSTNAME`. `CSRF_TRUSTED_ORIGINS` and `CORS_ALLOWED_ORIGINS` are set on a second pass, because the Vercel URL does not exist until the frontend is deployed.

Three free-tier limits — none of them bugs

Each of these first appeared as a confusing production-only symptom. When Aixa misbehaves only in production, check these before reading code.

- **Groq: 1000 output tokens/minute per organisation** — roughly one chat turn per minute across **all** users combined, not per user. Surfaced as an intermittent 429.
- **Gemini: 100 embedded chunks/minute.** A failed ingestion still spends the quota, so retrying early keeps the window permanently full. Ingest one document, then wait a clear minute. Surfaced as a document that could never index however often it was retried. The admin action detects `RESOURCE_EXHAUSTED/429` and replaces the raw JSON wall with that instruction.
- **Render free sleeps after 15 minutes idle**, and the next visitor pays a cold start of up to a minute. Whoever is actively testing keeps it warm, so the owner rarely sees the delay everyone else hits.

A proxy gotcha worth keeping

Next.js 308-redirects `/api/chat/stream/` to the slashless path **before** rewrites run; the browser re-POSTs, Next proxies a slashless path to Django, and `APPEND_SLASH` cannot redirect a POST without dropping the body — so it raises and every chat message 500s. `next.config.mjs` therefore sets `skipTrailingSlashRedirect: true` **and** ends the rewrite destination in `/`. The second half is required on its own: Next strips the trailing slash while matching, so `:path*` never carries it.

11. Testing

The split mirrors the architecture. `rag/` is tested with `pytest` (`pytest.ini` sets `pythonpath = apps`), needing no Django, database, API key, or network — `LangChain's GenericFakeChatModel` stands in for Groq. `chat/` is tested with the Django test runner, which needs the database and URL routing. Django app labels are `chat` and `rag`, not `apps.chat`.

```
cd backend
pytest apps/rag
python manage.py test chat --keepdb
```

- `test_f_chains_stream.py` — streaming returns docs unchanged and text with thinking stripped; `format_docs` numbers 1-based in retrieval order; all four suggestion paths (valid array, invalid JSON, non-list JSON, empty answer) return a list rather than raising.
- `test_g_stream_filter.py` — plain text passes through; a complete block is removed; a block whose tags split across chunks is removed; an unterminated trailing block is dropped.
- `test_stream_view.py` — full event ordering (sources, tokens in order, one suggestions event, done last); persistence of exactly one user and one assistant message with concatenated text; a bad session token rejected with 403 *before* any streaming; and the three regeneration cases.

The `tests/` directory at the repo root holds four early exploratory scripts, not automated tests. They reference paths that no longer exist and older function signatures — a record of how the pipeline was validated by hand, not a runnable suite.

Planned: Playwright end-to-end tests, Locust load testing, and RAGAS evaluation. RAGAS matters most, because it measures what unit tests cannot: whether retrieval surfaces the right chunks and whether an answer is faithful to them.

12. Security, Limitations, and Roadmap

What is protected

Signed session tokens make sequential session ids unguessable. `IsAdminUser` gates upload and ingestion, so a visitor cannot inject documents. Uploads are validated by extension and size. Both chat endpoints are throttled at 30/min. Startup refuses to run with debug off and no secret key. Both containers run as non-root users (`django`, `nextjs`). In production, HTTPS is enforced, HSTS is set for a year, and CSRF origins are explicit.

What is not

There is no authentication on chat, so anyone reachable can spend Groq quota; throttling bounds this without eliminating it. Session tokens live in `localStorage`, readable by any script on the page — acceptable for a public chatbot with no sensitive data, not for confidential conversations. Every retrieved chunk is sent to Groq and every question to Gemini, so ingesting a document discloses it in fragments to two external services. Uploaded PDFs are checked only for extension and size. The development compose file sets `ALLOWED_HOSTS` to `*` and allows all CORS origins — explicitly a development posture, overridden in `docker-compose.prod.yml`.

Known limitations

- **Ingestion is synchronous**, with no progress reporting or retry; a background queue is the fix.
- **PDF only**, with no OCR.
- **No API path to delete or re-ingest** a document's vectors; `is_ingested` is one-way.
- **Retrieval is fixed at $k=3$** , with no reranking, no hybrid keyword search, and no query rewriting — so a follow-up like "and what about that one?" retrieves poorly, since the pronoun carries no retrievable meaning even though history reaches the generation step.
- **History is capped at 20 messages**, sent verbatim with no summarisation.
- **Grounding is prompt-enforced, not verified** — no automatic check that a claim is supported by a retrieved chunk, and citation numbers are range-checked but not validated for correctness.
- **Answers cap at 700 output tokens**, so a long answer is truncated.
- **Feedback is local only** — thumbs up/down never reaches the backend, so it cannot inform evaluation.
- **No stop control** once a stream is in flight.

Roadmap

Done: the standalone RAG pipeline; Django and Postgres integration; upload and ingestion; the session-aware chat API; multi-turn context; the Next.js frontend; token-by-token streaming with server-side reasoning stripping; unit tests; Dockerised local development; and the live free-tier deployment on Vercel, Render, and Supabase with pgvector.

Next: Playwright end-to-end coverage, Locust load testing, and RAGAS retrieval and faithfulness evaluation. Optionally, air quality sensor data from the AeroBand project as a second retrieval domain alongside the CV.